Tech Rules (Tech Stack Definition)

The Himalaya Carbon Tokenization Engine will be developed as a **lightweight Minimum Viable Product (MVP)** designed for rapid prototyping, hackathon deployment, and startup demonstration.

The objective of the MVP is not to build a full national carbon infrastructure immediately, but to **demonstrate the core system logic**:

Registry-backed carbon unit → Tokenized representation → Market interface → Retirement verification.

To support fast development and clear demonstration, the technical stack prioritizes **simplicity, modularity, and widely adopted modern frameworks**.

3.1 Core Technical Principles

1. Simplicity First

The MVP avoids heavy DevOps infrastructure such as container orchestration or microservices. Development will use a **single full-stack framework** to reduce complexity and speed up development.

2. Web-Native Architecture

The application will be implemented as a **full-stack web application** using modern JavaScript frameworks. Both frontend and lightweight backend functionality will exist in the same codebase.

3. Blockchain as Infrastructure Layer

Blockchain components will only handle:

- token ownership
- minting
- transfer
- retirement verification

All other business logic (user accounts, metadata, UI dashboards) will remain in the web application layer.

4. Registry Integrity

Token creation must always correspond to **verified registry-backed carbon units**. Smart contracts must enforce permissioned minting to prevent unauthorized token issuance.

5. MVP-Ready Architecture

External integrations such as Bhutan's national carbon registry and CAD Trust APIs can initially be **simulated or mocked** for demonstration purposes.

3.2 Core Technology Stack

The Himalaya MVP will be built using the following technology stack.

Frontend Framework

Next.js

The platform frontend will be built using **Next.js**, a React-based full-stack framework.

Next.js enables:

- fast UI development
- server-side rendering
- integrated backend APIs
- simple deployment workflows

This framework allows the entire MVP to run within a **single application environment**, which significantly reduces development overhead.

Web3 Development Framework

Scaffold-ETH 2

The blockchain development environment will use **Scaffold-ETH 2**.

Scaffold-ETH 2 provides a preconfigured environment including:

- Next.js frontend integration
- smart contract deployment tools
- wallet connection infrastructure
- contract interaction utilities

Using Scaffold-ETH 2 accelerates development by removing the need to manually configure the Web3 stack.

This framework allows developers to quickly implement:

- token minting
- wallet interaction
- token transfers
- on-chain transaction display

without building the blockchain infrastructure from scratch.

Smart Contract Development

Solidity

Smart contracts will be written in **Solidity**, the standard programming language for Ethereum-compatible blockchains.

Solidity will be used to implement:

- carbon token minting
- token ownership tracking
- token retirement (burn function)
- permissioned administrative controls

Hardhat

Smart contract development and testing will use **Hardhat**, which is integrated within Scaffold-ETH 2.

Hardhat enables:

- contract compilation
- automated testing
- deployment scripts
- local blockchain simulation

OpenZeppelin Libraries

Smart contracts will utilize **OpenZeppelin standard libraries** to ensure secure and tested implementations of token standards.

These libraries provide secure implementations of:

- ERC-1155 multi-token standard
- access control
- pausable contract logic
- ownership management

3.3 Blockchain Network

For MVP deployment, the system will use a **public Ethereum-compatible test network**.

Recommended networks include:

- Polygon Amoy Testnet
  or
- Base Sepolia Testnet

These networks offer:

- low transaction cost
- reliable infrastructure
- compatibility with Ethereum tools
- simple wallet integration

The production system can later migrate to a mainnet environment or a government-approved blockchain network.

3.4 Token Architecture

The Himalaya system will use a **hybrid token architecture** designed specifically for carbon assets.

Canonical Carbon Tokens

The main token representation will use the **ERC-1155 multi-token standard**.

Each token ID represents a specific carbon credit category defined by:

- project
- vintage year
- methodology
- authorization status

Example token class:

Bhutan Forest Project – Vintage 2025 – Article 6 Authorized

ERC-1155 allows multiple token classes to be managed within a single smart contract while maintaining fungibility within each class.

Liquidity Tokens (Future Phase)

For decentralized exchange compatibility, token classes may later be wrapped into **ERC-20 tokens** to enable automated market maker (AMM) trading.

This feature is optional for the MVP and may be introduced in later phases.

3.5 Wallet Integration

User interaction with blockchain functions will require a wallet connection.

The platform will support wallet integration through:

- **RainbowKit**
- **Wagmi**

These tools allow users to connect common wallets such as:

- MetaMask
- WalletConnect-compatible wallets

Wallet functionality will be used for:

- token ownership
- transaction approval
- token retirement verification

However, the user interface will remain **web-first**, ensuring that the system feels like a normal platform rather than a cryptocurrency exchange.

3.6 Backend Logic

For the MVP, backend logic will be handled through **Next.js server APIs** rather than a separate backend server.

These APIs will manage:

- project metadata storage
- registry reference data
- transaction history
- retirement certificate generation
- user account data

This approach keeps the application architecture simple and easy to deploy.

3.7 Database

The system will use a relational database to store off-chain data.

Recommended database:

PostgreSQL

PostgreSQL will store:

- user accounts
- project metadata
- registry references
- retirement records
- audit logs

Prisma ORM

Prisma will be used as the database interface to simplify schema management and queries.

Database hosting may use platforms such as:

- Supabase
- Neon

These platforms allow quick setup and easy integration with Next.js.

3.8 User Interface

The platform interface will be built using:

- Tailwind CSS
- modern React components

This allows rapid development of clean, responsive dashboards including:

- carbon project listings
- token purchase interface
- retirement dashboard
- registry verification display
- admin minting interface

3.9 Deployment

The MVP will use lightweight cloud deployment tools suitable for startup projects.

Application Hosting

The web application will be deployed on **Vercel**, which provides seamless deployment for Next.js applications.

Smart Contract Deployment

Smart contracts will be deployed using Hardhat scripts to the selected blockchain testnet.

Database Hosting

PostgreSQL databases will be hosted using Supabase or Neon.

3.10 Development Structure

The project will use a **single code repository** containing both the frontend and blockchain components.

Typical structure:

/app
/components

/contracts
/lib
/prisma
/public

This unified structure simplifies development and allows rapid iteration during the MVP phase.

3.11 MVP Feature Scope

The initial MVP implementation will include:

- permissioned token minting
- ERC-1155 carbon token contract
- wallet connection
- carbon project dashboard
- token purchase simulation
- carbon retirement function
- retirement certificate generation
- admin management interface

Live integration with Bhutan's national registry and CAD Trust may be simulated for the initial demonstration.

3.12 Rationale for the Tech Stack

This stack was chosen because it enables:

- rapid development
- modern user interfaces
- reliable blockchain integration
- easy deployment
- scalability for future expansion

The architecture intentionally avoids complex infrastructure in order to keep the MVP **focused on demonstrating the core innovation**:

representing official Bhutan carbon registry units on-chain without compromising environmental accounting integrity.